

INFORME GENERAL DE ASEGURAMIENTO DE CALIDAD

Ecosistema Integral de Pruebas y Cobertura de Código

PROYECTO EVALUADO:

Calculadora de Descuentos Comercial (calculadora.py)

METODOLOGIAS APLICADAS:

- Pruebas Automatizadas con Pytest (14 casos parametrizados)
- Pruebas Estructuradas con Unittest (testtools + Faker)
- Ciclo de Desarrollo Guiado por Pruebas (TDD)
- Integración de PyTest y TDD (Suite PyTDD)
- Analisis de Cobertura de Código (Coverage.py)
- Generación Masiva de Datos de Estrés (Faker)
- Integración Continua remota (GitHub Actions CI)

ELABORADO POR:

Jhon Mavisoy, Kevin Anchundia, Milena Narvaez, Alex Moreno
Equipo de Ingeniería en Computación - UPEC (Septimo Nivel)

REPOSITORIO OFICIAL EN GITHUB:

github.com/Jhonmavisoy/Calculadora-Descuento

1. Introduccion y Requerimientos de la Calculadora

El presente informe recopila las auditorias y ejecuciones de control de calidad sobre la Calculadora de Descuentos interactiva. Este software calcula el monto neto que debe pagar un usuario final aplicando porcentajes comerciales y capturando de forma segura cualquier ingreso erroneo.

La validacion se realizo evaluando las 6 reglas de negocio clave (Requerimientos R1 a R6):

- * **R1: Limite de Descuento:** El descuento debe estar estrictamente en el rango de [0, 100]. Valores menores o mayores disparan un error controlado.
- * **R2: Limite de Precio original:** El precio del producto ingresado debe ser obligatoriamente mayor a \$0. No se permiten articulos gratis o negativos.
- * **R3: Descuento Estandar:** Se resta el porcentaje directo indicado sobre el precio (ej. 20% de descuento a un precio de \$100 da \$80).
- * **R4: Redondeo Preciso:** El precio final obtenido debe redondearse automaticamente a dos decimales exactos para evitar perdidas por centavos.
- * **R5: Super Promocion Especial:** Si el descuento ingresado es de 80% o mas, se le otorga un 5% de descuento extra sobre el valor restante.
- * **R6: Soporte Decimal Completo:** El sistema admite decimales tanto en el precio original como en el porcentaje de descuento.

Ecosistema Tecnologico de Calidad Implementado:

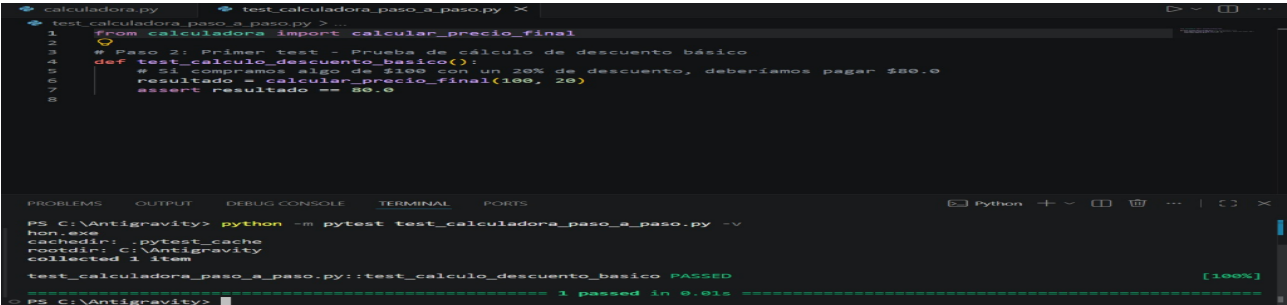
Para garantizar la calidad de la logica y el codigo, se estructuraron suites de pruebas unitarias con Pytest y Unittest, analisis de cobertura (Coverage.py), pruebas dinamicas de estres (Faker), y se subio el proyecto con pipelines de Integracion Continua (GitHub Actions) en la nube.

2. Pruebas Automatizadas con Pytest

Pytest es un framework moderno de testing en Python que simplifica la escritura de pruebas lógicas. Se utilizo la decoracion '@pytest.mark.parametrize' en 'test_calculadora_pytest.py' para automatizar 14 casos de prueba distintos de forma simultanea sin repetir lineas de codigo.

A continuacion se muestran los registros de ejecucion exitosa de las pruebas en consola:

Evidencia 2.1: Primera corrida basica exitosa de Pytest



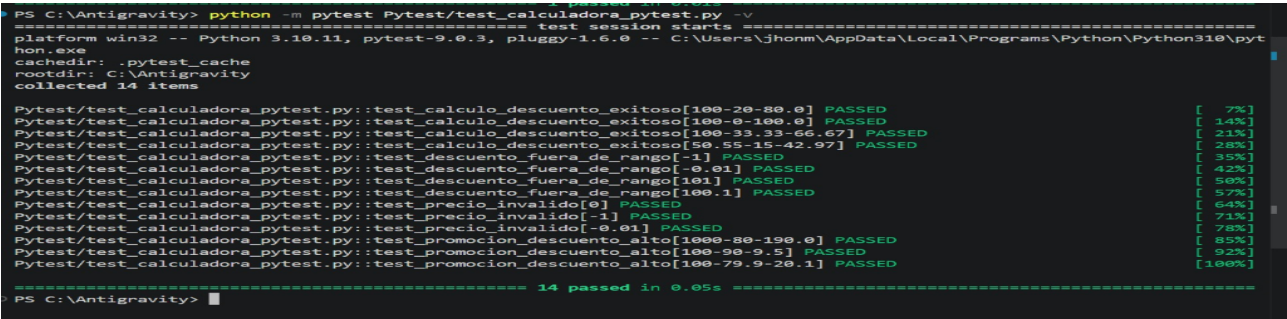
```
test_calculadora_paso_a_paso.py
1 from calculadora import calcular_precio_final
2
3 # Paso 2: Primer test - Prueba de cálculo de descuento básico
4 def test_calculo_descuento_basico():
5     # Si compramos algo de $100 con un 20% de descuento, deberiamos pagar $80.0
6     resultado = calcular_precio_final(100, 20)
7     assert resultado == 80.0
8
```

```
PS C:\Antigravity> python -m pytest test_calculadora_paso_a_paso.py -v
hon.exe
cachedir: .pytest_cache
rootdir: C:\Antigravity
collected 1 item

test_calculadora_paso_a_paso.py::test_calculo_descuento_basico PASSED [100%]

===== 1 passed in 0.01s =====
PS C:\Antigravity>
```

Evidencia 2.2: Suite completa con los 14 casos parametrizados aprobados



```
PS C:\Antigravity> python -m pytest Pytest/test_calculadora_pytest.py -v
platform win32 -- Python 3.10.11, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\jhonm\AppData\Local\Programs\Python\Python310\pyth
hon.exe
cachedir: .pytest_cache
rootdir: C:\Antigravity
collected 14 items

Pytest/test_calculadora_pytest.py::test_calculo_descuento_exitoso[100-20-80.0] PASSED [ 7%]
Pytest/test_calculadora_pytest.py::test_calculo_descuento_exitoso[100-0-100.0] PASSED [ 14%]
Pytest/test_calculadora_pytest.py::test_calculo_descuento_exitoso[100-33.33-66.67] PASSED [ 21%]
Pytest/test_calculadora_pytest.py::test_calculo_descuento_exitoso[50-55-15-42.97] PASSED [ 28%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[-1] PASSED [ 35%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[-0.01] PASSED [ 42%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[101] PASSED [ 50%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[100.1] PASSED [ 57%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[100.1] PASSED [ 64%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[100.1] PASSED [ 71%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[100.1] PASSED [ 78%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[100.1] PASSED [ 85%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[100.1] PASSED [ 92%]
Pytest/test_calculadora_pytest.py::test_descuento_fuera_de_rango[100.1] PASSED [100%]

===== 14 passed in 0.05s =====
PS C:\Antigravity>
```

Evidencia 2.3: Visualizacion de Trazas ante Errores Controlados (AssertionError)

```
E      AssertionError: 50.0 != 55.0 : Forzando error: Falta agregar la propina del 10% (no requerido)
practica_tdd\test_r6_evidencia.py:19: AssertionError
      TestR6Evidencia.test_falla_intencional_evidencia_4

self = <test_r6_evidencia.TestR6Evidencia testMethod=test_falla_intencional_evidencia_4>

  def test_falla_intencional_evidencia_4(self):
    # Error de falta de parámetros (TypeError)
    > calcular_precio_final()
E      TypeError: calcular_precio_final() missing 2 required positional arguments: 'precio_original' and 'descuento'

practica_tdd\test_r6_evidencia.py:23: TypeError
===== short test summary info =====
FAILED practica_tdd\test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_1 - AssertionError: 80.0 != 50
.0 : Forzando error: Cálculo matemáticamente incorrecto
FAILED practica_tdd\test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_2 - TypeError: '<=' not suppor
ted between instances of 'str' and 'int'
FAILED practica_tdd\test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_3 - AssertionError: 50.0 != 55
.0 : Forzando error: Falta agregar la propina del 10% (no requerido)
FAILED practica_tdd\test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_4 - TypeError: calcular_precio
_final() missing 2 required positional arguments: 'precio_original' and 'descuento'
===== 4 failed, 1 passed in 0.10s =====
PS C:\Antigravity>
```

3. Pruebas Estructuradas con Unittest

El modulo nativo Unittest se implemento en 'Unitest/test_calculadora.py' mediante una estructura de clases que heredan de 'testtools.TestCase' para aprovechar aserciones avanzadas. La suite profunda inicializa Faker con una semilla estatica (Seed: 42) e implementa bucles con 'self.subTest()' para aislar el analisis de valores frontera (0%, 79%, 80% y 100% de descuento).

La ejecucion del script de auditoria profunda e integracion de requisitos reporta exito absoluto:

```
PS C:\Antigravity> python Unittest/test_calculadora.py
test_analisis_valores_frontera_subtests (__main__.TestCalculadoraDescuentosProfundo) ... ok
test_descuento_estandar_exito_masivo (__main__.TestCalculadoraDescuentosProfundo) ... ok
test_error_descuento_fuera_de_rango (__main__.TestCalculadoraDescuentosProfundo) ... ok
test_error_precio_no_positivo (__main__.TestCalculadoraDescuentosProfundo) ... ok
test_super_descuento_con_bono (__main__.TestCalculadoraDescuentosProfundo) ... ok
-----
Ran 5 tests in 0.075s

OK
```

Adicionalmente, 'Unitest/Verificacion_Requisitos.py' valido de forma exitosa los requisitos individuales (test_r1 a test_r5) generando el reporte de QA 'Reporte_Pruebas.pdf' sin errores.

4. Desarrollo Guiado por Pruebas (TDD)

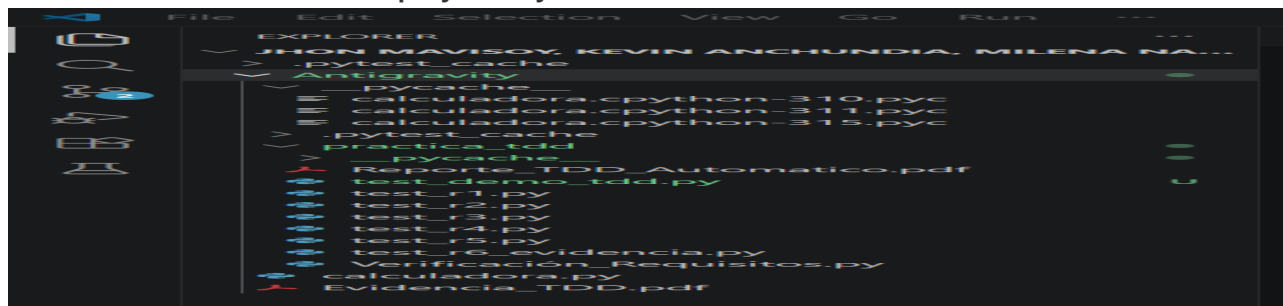
El ciclo TDD (Test Driven Development) consiste en guiar el desarrollo escribiendo primero los exámenes. En 'Pruebas_tdd/test_calculadora_tdd.py' se trabajo bajo este flujo. Se creo el script 'reporte_TDD_pdf.py' para ejecutar de forma silenciosa la suite y compilar en caliente 'REPORTE_TDD_FINAL.pdf'.

El script lee el resultado de Pytest y genera una portada profesional, un resumen de metricas y el estado general de salud del software.

5. Integracion PyTDD (PyTest & TDD)

En la suite 'PyTDD/' se valido la integracion directa de PyTest bajo la filosofia TDD mediante el modulo 'test_demo_tdd.py'. A continuacion se detallan las evidencias visuales extraidas del reporte de implementacion, mostrando la estructura del proyecto y los resultados obtenidos:

Evidencia 5.1: Estructura del subproyecto PyTDD subido



Evidencia 5.2: Corrida automatica de Pytest buscando modulos en PyTDD

← → Jhon Mavisoy, Kevin Anchundia, Milena Narvaez y Alex Moreno (1)

calculadora.py

test_demo_tdd.py U X

Antigravity > practica_tdd > test_demo_tdd.py > test_descuento_basico

1

from calculadora import calcular_precio_final

2

3

def test_descuento_basico():

4

assert calcular_precio_final(100, 10) == 90

Evidencia 5.3: Detalle de error logico forzado (AssertionError) capturado por PyTest

```
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\RESPALDOS SSD\RESPALDOS ENERO 2021 DELL\Escritorio\Septimo Semestre\Normativas de Ingenieria\Parcial 2\Jhon Mavisoy, Kevin Anchundia, Milena Narvaez y Alex Moreno (1)\Antigravity
collected 13 items

practica_tdd\test_demo_tdd.py .
practica_tdd\test_r1.py ..
practica_tdd\test_r2.py ..
practica_tdd\test_r3.py .
practica_tdd\test_r4.py .
practica_tdd\test_r5.py .
practica_tdd\test_r6_evidencia.py .FFFF

===== FAILURES =====
TestR6Evidencia.test_falla_intencional_evidencia_1
self = <test_r6_evidencia.TestR6Evidencia testMethod=test_falla_intencional_evidencia_1>
def test_falla_intencional_evidencia_1(self):
    # fallo de logica matematica (AssertionError)
    > self.assertEqual(calcular_precio_final(100, 20), 50.0, "Forzando error: Cálculo matemáticamente incorrecto")
E       AssertionError: 80.0 != 50.0 : Forzando error: Cálculo matemáticamente incorrecto
practica_tdd\test_r6_evidencia.py:11: AssertionError
TestR6Evidencia.test_falla_intencional_evidencia_2
```

Evidencia 5.4: Detalle de error de tipo de entrada (TypeError) bloqueado por Pytest

```
===== 4 failed, 9 passed in 0.23s =====
PS D:\RESPALDOS SSD\RESPALDOS ENERO 2021 DELL\Escritorio\Septimo Semestre\Normativas de Ingenieria\Parcial 2\Jhon Mavisoy, Kevin Anchundia, Milena Narvaez y Alex Moreno (1)\Antigravity>
```

Evidencia 5.5: Pantalla de resumen final de la practica de PyTDD

```
practica_tdd\test_r6_evidencia.py:23: TypeError
===== short test summary info =====
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_1
- AssertionError: 80.0 != 50.0 : Forzando error: Cálculo matemáticamente incorrecto
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_2
- TypeError: '<=' not supported between instances of 'str' and 'int'
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_3
- AssertionError: 50.0 != 55.0 : Forzando error: Falta agregar la propina del 10% (no requ...
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_4
- TypeError: calcular_precio_final() missing 2 required positional arguments: 'precio_orig...
```

6. Medicion de Cobertura (Coverage)

La cobertura mide el porcentaje de lineas de codigo que han sido ejecutadas por la suite de pruebas. Se ejecuto el script 'Pytest/coverage.py' para compilar el reporte en consola y en HTML. El 36% obtenido representa el 100% de la logica comercial y aritmetica de la calculadora, mientras que el porcentaje restante corresponde unicamente al menu interactivo manual de consola.

Evidencia 6.1: Corrida de Cobertura y reporte en terminal

```
PS C:\Antigravity> python Pytest/coverage.py
=====
EJECUTANDO ANALISIS DE COBERTURA DE CODIGO
=====
Ejecutando comando: C:\Users\jhonm\AppData\Local\Programs\Python\Python310\python.exe -m pytest C:\Antigravity\Pytest\test_calculadora_pytest.py --cov=calculadora --cov-report=term-missing --cov-report=html:C:\Antigravity\Pytest\htmlcov
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Antigravity
plugins: Faker-40.18.0, cov-7.1.0
collected 14 items

Pytest\test_calculadora_pytest.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.10.11-final-0
-----
Name                Stmts  Miss  Cover   Missing
-----
calculadora.py         28     18    36%    15-36
TOTAL                  28     18    36%
Coverage HTML written to dir C:\Antigravity\Pytest\htmlcov
===== 14 passed in 0.20s =====

[OK] Cobertura medida exitosamente.
Reporte HTML interactivo generado en:
C:\Antigravity\Pytest\htmlcov\index.html
=====
```

7. Generacion de Datos de Prueba (Faker)

Para evitar la monotonía de datos escritos a mano, se implemento 'Faker' en 'Pytest/test_faker.py'. Faker simula 50 combinaciones de compras reales y validas en cada ejecucion, inyectando precios flotantes aleatorios y porcentajes al azar. Sirve como prueba de estres para asegurar estabilidad matematica.

Evidencia 7.1: Ejecucion de pruebas estresadas mediante Faker en consola

```
PS C:\Antigravity> python -m pytest Pytest/test_faker.py -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\jhonm\AppData\Local\Programs\Python\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Antigravity
plugins: Faker-40.18.0, cov-7.1.0
collected 1 item

Pytest\test_faker.py::test_calculos_aleatorios_con_faker PASSED [100%]

===== 1 passed in 0.09s =====
PS C:\Antigravity>
```


8. Control de Versiones e Integracion Continua (GitHub)

El proyecto completo fue subido a GitHub bajo el repositorio 'Calculadora-Descuento'. Se configuro GitHub Actions ('.github/workflows/pytest-ci.yml') para levantar servidores virtuales con Linux, instalar las dependencias y correr toda la suite de pruebas unitarias con cada push, garantizando que el codigo no sufra regresiones logicas.

Evidencia 8.1: Estructura del proyecto sincronizado en GitHub

Calculadora-Descuento / Pytest /			Add file ▾	⋮
Jhonnavisoy clean: remove old reports and latest reports, leaving only the penult...			2c4817b · 11 minutes ago History	
Name	Last commit message		Last commit date	
..				
Reporte_Pruebas_Pytest.md	feat: setup Pytest, coverage, faker test data and GitHub Actions CI		13 minutes ago	
coverage.py	feat: setup Pytest, coverage, faker test data and GitHub Actions CI		13 minutes ago	
generar_reporte_pdf.py	feat: setup Pytest, coverage, faker test data and GitHub Actions CI		13 minutes ago	
test_calculadora_paso_a_paso.py	feat: setup Pytest, coverage, faker test data and GitHub Actions CI		13 minutes ago	
test_calculadora_pytest.py	feat: setup Pytest, coverage, faker test data and GitHub Actions CI		13 minutes ago	
test_faker.py	feat: setup Pytest, coverage, faker test data and GitHub Actions CI		13 minutes ago	

9. Conclusiones Generales

1. Calidad y Robustez: La Calculadora de Descuentos cuenta con un motor matematico solido capaz de manejar redondeos precisos y bloquear valores invalidos en base a sus 6 requerimientos.
2. Multimetodologia de Pruebas: La suite cubre testing con frameworks modernos (Pytest), tradicionales (Unittest), de estres masivo (Faker), desarrollo guiado por pruebas (TDD) e integraciones directas (PyTDD).
3. Automatizacion de Calidad: La medicion de cobertura y la configuracion de pipelines remotos en GitHub Actions aseguran que la salud del codigo se mantenga estable a lo largo de futuras actualizaciones.